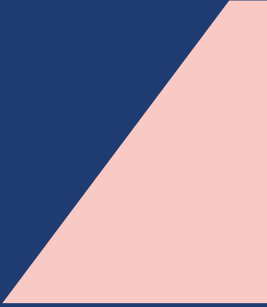


Community Security Alert System

System Simulation and Validation

Felipe Jose Garzon Herrera	20251020132
Juan Esteban Quintero Gordillo	20251020137
Henry Samuel Garrido Medina	20251020125
Gabriel Mateo Cusba Marin	20251020128

1. Introduction
2. Simulation Paradigm
3. System Architecture
4. Requirements Coverage
5. Parameters and Experimental Design
6. Risks and Mitigations
7. Results
8. Statistical Validation
9. Emergent Behavior
10. Contingency Plan
11. Design Validation
12. Conclusions



Introduction



Context and Purpose

! Problem

High incidence of theft and vandalism on campus with 35 % under-reporting and only 12 % initial adoption.

© Workshop Purpose

Evaluate CSAS operational behavior under different campus security conditions through computational simulation, testing the proposed architecture in a controlled environment.

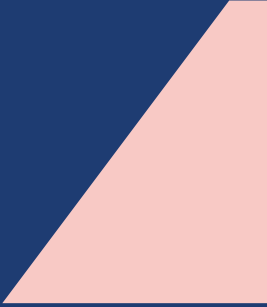
Workshop	Focus
W1	Security problems
W2	System architecture
W3	Risk management
W4	Simulation & validation

DES

Behav.

3 Scen.

30 runs



Simulation Paradigm



Discrete Event Simulation (DES)

- Models asynchronous events: reports, verification, dispatch
- Implemented with **SimPy** in Python
- Advances event-by-event (not continuous)
- Efficient for queues, delays, and concurrency

Modeled Events

1. User submits incident report
2. Verification Service processes
3. Dispatcher broadcasts alert
4. Analytics records metrics



Python + SimPy library



Behavioral Simulation

Complements DES by modeling community response through two feedback loops:

+ Reinforcing Loop (R^+)

Verified alert $\rightarrow$ trust $\uparrow$ $\rightarrow$ vigilance $\uparrow$ $\rightarrow$ more reports $\rightarrow$ self-sustaining cycle

- Balancing Loop (B^-)

Too many false alerts $\rightarrow$ fatigue $> 60\%$ $\rightarrow$ adoption drops 4% per step $\rightarrow$ collapse

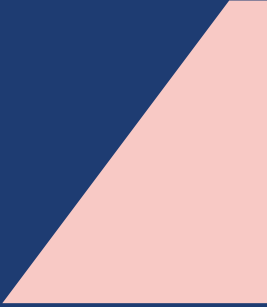
- 12 simulation steps (15:00–02:00)
- Environmental multipliers by hour, weather, zone
- Population: 200 pilot users

Key Finding

Verification threshold θ

determines which loop dominates

$\theta \geq 3 \Rightarrow$ stable system

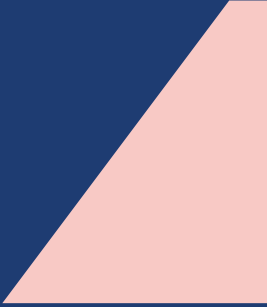


System Architecture

Module	Responsibility
User Process	Generates incidents dynamically
Incident Svc	Receives and queues reports
Verification Svc	AI + community verification

Module	Responsibility
Dispatcher Svc	Geofenced alerts (≤ 500 m)
Analytics Mod.	Performance metrics

$\text{verified} = (\text{confidence} \geq 0.75) \wedge (\text{confirmations} \geq \theta \vee \text{admin approval})$



Requirements Coverage



Functional and Non-Functional Requirements



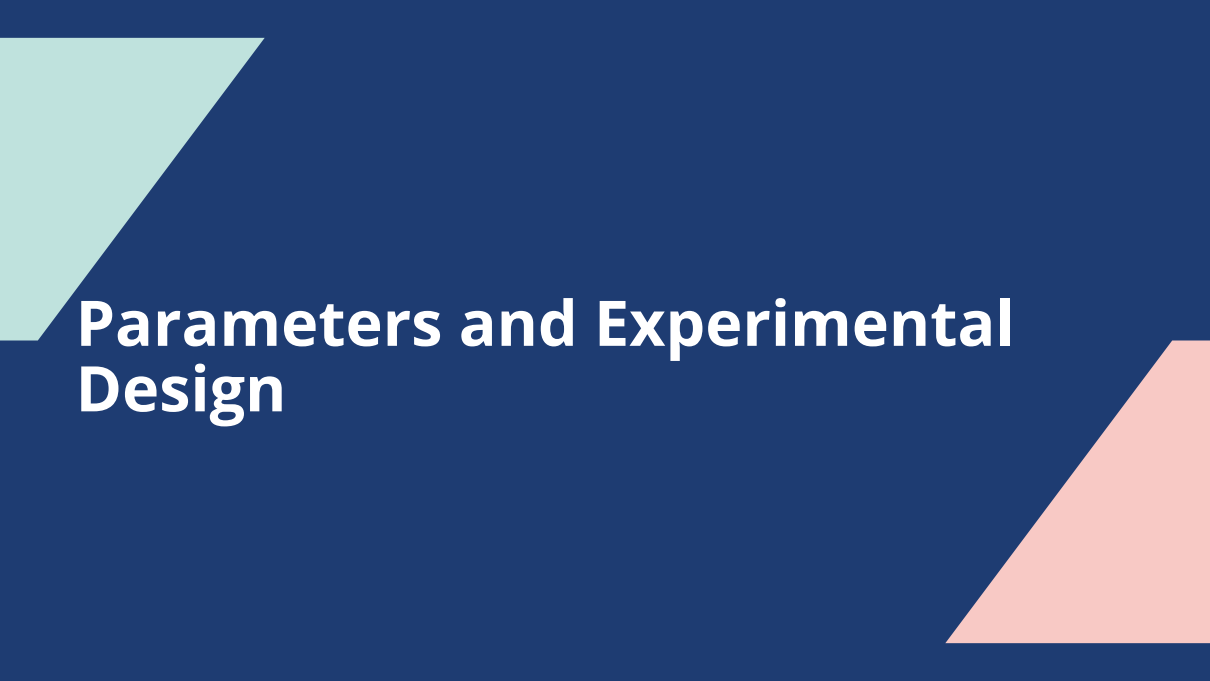
Functional Requirements

FR	Simulation
FR-01	Users generate incidents
FR-02	Geofenced alert 500 m
FR-03	AI validation ≥ 0.75
FR-04	Console + CSV export
FR-05	Random confirmations [0,4]
FR-09	Metrics per step



Non-Functional Requirements

NFR	Mechanism
NFR-01	Latency < 30 s
NFR-02	Failure prob. $p = 2\%$
NFR-05	Concurrent users
NFR-06	≤ 3 interactions
NFR-07	Modular classes
NFR-08	Dual verification gate

The background is a solid dark blue. In the top-left corner, there is a light blue triangle pointing towards the center. In the bottom-right corner, there is a pink triangle pointing towards the center.

Parameters and Experimental Design



Simulation Configuration



DES

Parameter	Value
Sim. time	200 s
Users	11
Incident interval	20–30 s
Verif. time	1–2 s
Dispatch time	1–5 s
Failure prob.	2 %



Behavioral

Parameter	Value
Steps	12 (15:00–02:00)
Population	200
Geofence	500 m
Seed	42 (fixed)
Runs/scenario	30



Metrics

- Processed incidents
- Verified alerts
- Failed deliveries
- Average latency
- % compliance < 30 s
- Adoption rate



Experimental Design

EXP	Hypothesis	IV	DV (Target)
EXP-01	Traffic peaks degrade latency	Load 100–300 %	Latency < 30 s
EXP-02	Validation saturates staff	Peak complexity	E2E latency < 60 s
EXP-03	Low reporting culture	UI friction	Adoption $\geq$ 60 %

☑ Protocol

Fixed seed → 12 steps → Paired baseline vs opt.

🏗 3 Scenarios

Baseline / Optimization / Failure Mode



Risks and Mitigations



Risk Register (R1–R10)

ID	Risk	Prob.	Impact
R1	System overload	High	Critical
R2	Notification failure	Medium	High
R3	False reports	High	High
R4	Data breach	Low	Critical
R5	Low adoption	Medium	High
R6	Connectivity loss	High	Medium
R7	API integration failure	Medium	High
R8	Slow authority response	Medium	High
R10	Database failure	Low	Critical



Mitigation Strategies

M1 — Verification Threshold θ

With $\theta \geq 3$ filters false positives without exceeding latency.

$$\text{latency} = \frac{r \times 15}{\text{eff.}} < 60 \text{ s}$$

⚠ Affects: R1, R2, R3

M2 — Critical Adoption Mass

Minimum 30–35 % adoption. UX ≤ 3 taps + progressive onboarding.

⚠ Affects: R5, R9

M3 — Alert Fatigue Prevention

Fatigue $> 60\%$ $\Rightarrow$ adoption drops 4 %/step (irreversible).

$\theta \geq 3$ prevents it.

⚠ Affects: R6, R8

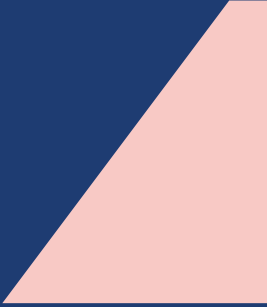
M4 — Security and Resilience

TLS 1.3, AES-256, circuit breakers, multi-zone replication, backups every 15 min.

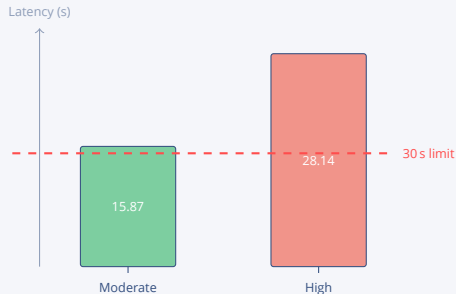
✅ Affects: R4, R7, R10



Results



DES Results — Moderate vs. High Load



 Average latency comparison

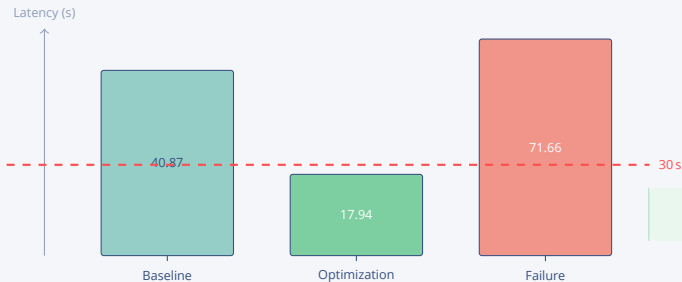
Moderate Load

Metric	Value
Incidents	23
Failed alerts	0
Users reached	30
Avg. latency	15.87 s
% $\leq$ 30 s	100 %

High Load

Metric	Value
Incidents	29
Failed alerts	1
Users reached	47
Avg. latency	28.14 s
% $\leq$ 30 s	55.17 %

Behavioral Results — 30 Runs (95 % CI)



Scenario	Latency	Adoption
Baseline	40.87 s	21.39 %
Optimization	17.94 s	67.62 %
Failure Mode	71.66 s	31.34 %

✓ Optimization: CI ± 1.02 s $\Rightarrow$ consistent



Statistical Validation

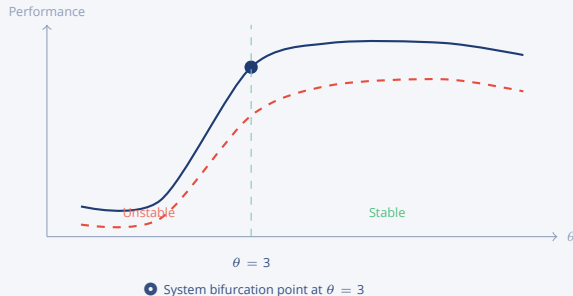
Confidence Intervals

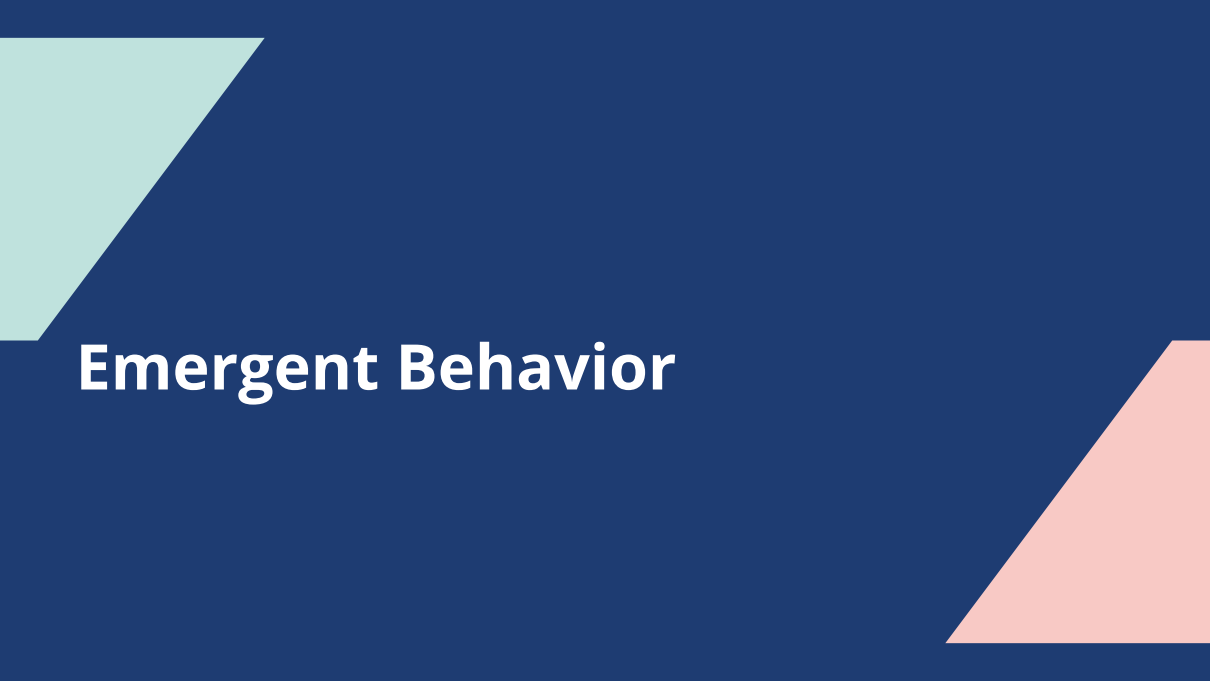
$$\bar{x} \pm t_{29,0.025} \cdot \frac{s}{\sqrt{30}}, \quad t_{29,0.025} = 2.045$$

Narrow CIs $\Rightarrow$ stable and reproducible results.

Most Influential Variables

1. **Threshold θ :** from 1 to 3 reduces false positives $\approx 78\%$
2. **Staff efficiency:** 50% $\rightarrow$ 90% halves latency

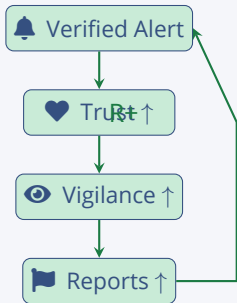




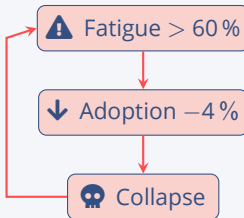
Emergent Behavior



Feedback Loops



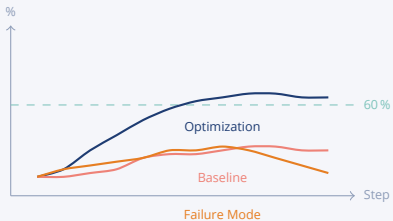
✓ $\theta \geq 3$: R+ dominates (stable)



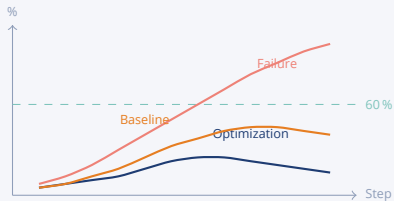
✗ $\theta = 1$: B- dominates (collapse)

Temporal Evolution — Fatigue and Adoption

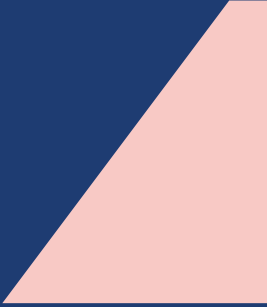
Community Adoption



Alert Fatigue



 Fatigue grows exponentially in Failure Mode; Optimization keeps it in safe range



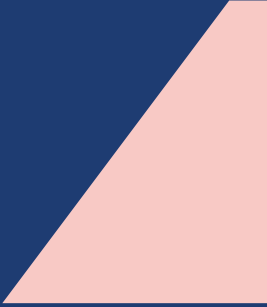
Contingency Plan



Failure Response

Scenario	Trigger	Immediate Action	Mechanism
Overload (R1)	Latency > 30 s	Scale K8s pods	HPA + PagerDuty
Notif. failure (R2)	Delivery < 95 %	Activate SMS fallback	RabbitMQ DLQ + Twilio
Fatigue (R3)	Fatigue > 60 %	$\theta \rightarrow 4$; pause non-critical	Feature flag
Data breach (R4)	SIEM anomaly	Isolate service; revoke tokens	Circuit breaker
DB failure (R10)	Primary unreachable	Promote replica	Multi-AZ failover

🕒 All responses execute within 30 s of trigger detection



Design Validation

Workshop 1 — Findings

- 35 % incident under-reporting
- 12 % initial adoption
- Identified risk zones and hours

Challenged Assumption

12 % initial adoption is insufficient. Structured onboarding campaign required before Phase 1 deployment.

Workshop 2 — Architecture

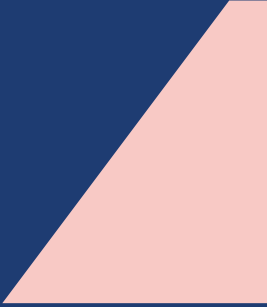
- ✓ RabbitMQ + K8s validated (EXP-01)
- ✓ Automated engine (EXP-02)
- ✓ ≤ 3 tap flow (EXP-03)
- ✗ 12 % adoption unsustainable

Workshop 3 — Risks

- 8/10 risks quantitatively validated
- R1 and R6 observed in Failure Mode



Conclusions





Conclusions

1. Verification threshold θ is the most important parameter.

- $\theta \geq 3$: adoption $> 60\%$, latency within NFR-01
- $\theta = 1$: irreversible system collapse

2. Technology alone is not sufficient.

- Without 30–35 % community adoption, the system cannot sustain itself

3. Workshop 2 architecture is validated.

- RabbitMQ, K8s, automated engine, and SMS fallback meet targets



Recommendations

Short Term

1. $\theta = 3$ as default
2. Onboarding $> 35\%$
3. Fatigue dashboard

Medium Term

1. Trained classifier
2. Real GPS data
3. > 12 simulation steps

Long Term

1. Agent-based modeling
2. SIURE integration
3. Gamification

Questions?

 github.com/filipghs/Systems-Analysis

 Workshop 4 Simulation / W4_DES.py & W4_behavioral.py

Community Security Alert System